System Design Reflection 2

Rokas Pazera 551495

Design and Implementation

During weeks 3 and 4, I started working on UML class diagrams. I had to learn new rules like visibility symbols, static vs final, and different relationship types like association, aggregation, composition, dependency, inheritance, and realization. I used the *Grading* domain classes (*Person, Student, Teacher, Coach, Grade, Course*) and made the diagram straight from the real Java code, so everything in the diagram had to match the real code.

Most of the changes happened because I kept checking the diagram against the code. First, I used a filled diamond for *Grade* to *Student* and *Course* to *Teacher*, but this should be a normal association arrow. Then, I used a hollow diamond for *Person* to *Student* and *Person* to *Teacher*, but this should be an inheritance triangle. I also forgot the dashed arrow for *Teacher* and *Coach* implementing *Payable*, and I even had a salary field on *Coach* that doesn't even exist in the real class. After fixing this one by one, I understood how different composition, aggregation, inheritance, and realization really are, and that I should also double check the diagram contents to avoid adding nonexisting variables or methods.

In week 4 I used what I learned to make 2 new designs instead of fixing an old one: a data model for a pokemon card game, and a *NotificationService* that follows the Open/Closed principle. For the card model, the big decision was to use composition between *Card* and *Ability*, because an *Ability* can only exist inside one card. For the *NotificationService*, I made it depend on an interface *NotificationSender* instead of a real class. This means if I want to add a new type, like *Telegram*, I only need to add one new class, instead of changing anything in the existing code.

Collaboration

I worked individually again this period. The downside showed up when I made the same diamond versus triangle mistake twice in a row - once for Person to Student and again for Person to Teacher. If there was a classmate looking at the diagram, they probably would have spotted that mistake and corrected me.

Research and Professional Development

The skill gap from weeks 1 and 2 was UML notation and in week 3 and 4 I saw that this gap was bigger than I thought. Just knowing the name of each arrow wasn't enough, because I still used them wrong:

- I mixed up composition (filled diamond) and inheritance (triangle) more than once. I think I picked the wrong one because I looked at what feels like “ownership”, instead of checking if it is really **HAS A** or **HAS A AND CANNOT EXIST WITHOUT.**

- I learned the Strategy pattern as a real way to use the Open/Closed principle. If a class depends on an interface instead of one specific class, it’s possible to add new behavior with a new class instead of if/else statements.

To get better at this, I checked the real source code every time instead of trusting how the diagram looked. I also wrote some diagrams in PlantUML instead of draw.io, because PlantUML makes you type the real relationship type, so it's harder to use the wrong one by accident.

Progress and Insights

My last reflection said I want to finish a diagram only when every arrow and color is correct. I tried to do this in week 3 and 4, but it wasn't enough, because I still used the wrong relationship type more than once. This happened because I was checking if the diagram looked right instead of checking every line with a clear rule.

In the next period I want to go through every relationship in a class diagram and ask one question for each line – is this

- **HAS A,**
- **HAS A AND CANNOT EXIST WITHOUT,**
- **HAS A BUT CANNOT EXIST ALONE,**
- **JUST A REFERENCE,**
- **ONLY USED ONE TIME.**

If I cannot answer this in one sentence, I know I need to check the code again before I finish the arrow.